Projects
Machine Learning Course
Fall 2026

Westlake University
Department of Artificial Intelligence, SOE
Tao Lin
https://github.com/LINs-lab/course_machine_learning

Class Project 2
Submission Deadline: Dec 24, 2026

Submission instructions:

- Please submit a single `.zip` archive to Canvas (`https://canvas.westlake.edu.cn/courses/2063`). The archive must contain your complete runnable codebase, the written report (the compiled `main.pdf` and all LaTeX sources under `submission/`), run logs, seed lists, the self-edit ledger, and all assets required to reproduce your headline results from scratch.
- Deadline: 23:59 on Dec 24, 2026 (Beijing time, UTC+8, sharp).

Introduction

Welcome to Project 2! In this project, you will study and build strategies for an autonomous research harness: a **Recursive Self-Improvement (RSI)** system named `Autoresearch-Base`.

Modern automated machine learning and autonomous research systems rely on iterative loops: formulating hypotheses, running empirical trials, updating a persistent body of knowledge, and deciding subsequent actions. A truly recursive self-improving system goes one step further: in addition to refining solutions to external tasks, the system can rewrite its *own* search and proposal strategies under a verified trust boundary, feeding those modifications directly back into subsequent cycles.

In this project, you are provided with a complete, fully functional research harness written in Python. It executes deterministically on CPU in seconds with no external API keys required. However, the system ships with deliberately **naive** strategies: it explores parameter spaces blindly, ignores accumulated evidence, and quickly hits performance plateaus.

Your assignment is to build the intelligence: design, motivate, implement, and evaluate principled strategies for experiment proposal, guarded self-improvement, result diagnosis, benchmark evolution, and knowledge-graph reasoning. You will empirically demonstrate that your system achieves measurable yield improvements across five frozen evaluation tasks.

Grading. Project 2 counts 30% toward your final grade in the course (scored out of 100 points according to an assessment guide detailed in the *Grading Criteria and Assessment Guide* section).

Logistics

Individual work. Project 2 is an individual project. You must work entirely **on your own**. Collaborative coding, sharing strategy implementations, or exchanging submission materials is strictly prohibited and constitutes academic misconduct.

Submission deadline. Dec 24, 2026 (at 23:59 Beijing time, UTC+8, sharp).

Deliverables at a glance.

- **Written Report.** A comprehensive scientific report of at most **9 pages of body text** (excluding references, checklist, and appendix), typeset using the official NeurIPS 2025 preprint template provided in the `submission/` directory.

- **Code and Reproducibility Archive.** A single `.zip` file submitted via Canvas containing:
  - The complete Python codebase with all your strategy implementations behind the defined `Protocol` seams.
  - The `submission/` folder containing `main.pdf` and its complete LaTeX sources.
  - Full experimental records: run logs, session summaries (`scripts/report.py` outputs), seed lists, and the self-edit ledger (`state/ledger.jsonl`).
  - Exact reproduction instructions in `README.md` and the environment definition in `environment.yml`.

System Overview: Autoresearch-Base

The `Autoresearch-Base` system is organized around three nested operational loops, coupled with a guarded recursive self-improvement circuit:

- **Fast Loop (Cycle-by-cycle):** Proposes, implements, runs, finalizes, and analyzes single-variable experiments on one research question under a frozen benchmark epoch:

  $$\text{propose} \longrightarrow \text{implement} \longrightarrow \text{run} \longrightarrow \text{finalize} \longrightarrow \text{analyze}$$

  The proposer reads the strategy artifact (`state/strategy.py`), stages an isolated working directory, dispatches the task entrypoint with injected benchmark metadata, evaluates outcomes against verdict gates (`keep` | `discard` | `fail`), ratchets the incumbent solution on keeps, and appends structured machine evidence to the SQLite knowledge store.

- **Middle Loop (Every N cycles):** Aggregates multi-round evidence, diagnoses saturation, noise floors, and metric discrimination, manages benchmark epochs (freezing within an epoch and upgrading across epochs), generates efficiency reports on wasted compute, and arbitrates live self-edits.

- **Slow Loop (Question-graph evolution):** Evolves a typed Research Question Graph using schema-validated contracts (I-Q questions, I-H hypotheses, I-E evidence, I-S insights, and I-K theory). When a hypothesis reaches a terminal state, a successor hypothesis is mounted on the same question; when a question saturates or exhausts its proposal space, the slow loop branches along typed edges (`refines`, `depends_on`, `challenges`, `spawns`).

- **System-Level Self-Improvement (The RSI Circuit):** Beyond optimizing external task code, the engine may propose modifications to its *own* proposal strategy inside a guarded sentinel region in `state/strategy.py`. An edit moves through the lifecycle:

  $$\text{applying} \longrightarrow \text{observing} \longrightarrow \text{kept} \mid \text{reverted}$$

  A promoted (`kept`) edit **re-enters immediately** because the proposer re-reads the strategy file every cycle. That feedback loop—waste detection $\rightarrow$ guarded self-edit $\rightarrow$ re-entry $\rightarrow$ measured yield increase $\rightarrow$ promotion—is the essence of recursive self-improvement.

The Trust Boundary (C/F Split)

To maintain scientific validity, the harness strictly separates mutable strategy surfaces from the components that evaluate performance:

1. **Strict Surface Allowlist:** The self-improver may edit only the sentinel region of `state/strategy.py` (`EDITABLE_SURFACES`).
2. **Executable Guard Layers (0–4):** All self-edits must pass multi-layer safety verification:
   - *Layer 0 (Surface allowlist):* Rejects edits outside `state/strategy.py`.
   - *Layer 1 (Sentinel integrity):* Ensures exactly one valid mutable sentinel region exists.
   - *Layer 2 (Byte identity):* Enforces that all code outside sentinels remains bit-for-bit identical.
   - *Layer 3 (Statement shape lock):* Locks top-level statement structure; assignment values may vary within their types, but statements cannot be added, deleted, retyped, or hijacked.

Task	Metric (Opt)	Baseline	RSI OFF	RSI ON (Naive)	Claude Code (Mean ± SD)
Battle of the Sexes	Mean Payoff (↑)	1.342	1.523	1.499	**1.690 ± 0.000**
Prisoner's Dilemma	Mean Payoff (↑)	1.680	2.743	2.743	**2.792 ± 0.000**
Colonel Blotto	Value Rate (↑)	0.528	0.599	0.599	**0.767 ± 0.017**
House Price	RMSE (↓)	14,517	11,552	11,779	**9,507 ± 14.44**
Breakout Lite	Bricks Cleared (↑)	6.42	53.42	53.25	**60.00 ± 0.000**

Table 1: Public evaluation task baselines under the frozen 20-minute budget protocol (task seed 7). Scores represent harness-verified best values.

- *Layer 4 (Override scan):* An independent scan/judge defaulting to rejection upon any ambiguity.

3. **Frozen Scorer Guarantee:** Task scorers (`run.py`), verdict gates, diagnostics, and knowledge schemas are completely frozen by construction. Any attempt to modify evaluation files, bypass gates, or tamper with benchmark scoring is caught by the test harness, results in a grade of zero, and is treated as academic misconduct.

Evaluation Tasks and Baselines

Task Bundles

Each research task bundle is a self-contained directory governed by a `program.md` manifest. It specifies an entrypoint, exactly one editable file, a frozen gate metric, and a fixed compute budget. Graded evaluations are performed on pristine copies of five public tasks:

1. **Battle of the Sexes** (`tasks/battle-of-sexes`): A 2-player coordination game with conflicting preferences. Editable file: `strategy.py`. Metric: **Mean Payoff** (direction: maximize).
2. **Prisoner's Dilemma** (`tasks/prisoners-dilemma`): Iterated social dilemma with temptation and betrayal. Editable file: `strategy.py`. Metric: **Mean Payoff** (direction: maximize).
3. **Colonel Blotto** (`tasks/colonel-blotto`): Resource allocation game across multiple battlefields. Editable file: `strategy.py`. Metric: **Value Rate** (direction: maximize).
4. **House Price** (`tasks/house-price`): Tabular regression predicting house prices under noisy real-world features. Editable file: `model.py`. Metric: **RMSE** (direction: minimize).
5. **Breakout Lite** (`tasks/breakout-lite`): Reinforcement learning control for paddle tracking and brick clearance. Editable file: `policy.py`. Metric: **Bricks Cleared** (direction: maximize).

In addition, two fast development bundles are provided: `tasks/quad-optim` (an ill-conditioned quadratic demo finishing in milliseconds) and `tasks/iris-ood` (out-of-distribution generalization under a seeded feature warp, with a rich reference experiment in `experiments/2026-10-11-iris-ood/`).

The Public Baseline Exhibit

The benchmark protocol is standardized across all tasks: frozen task seed 7, a uniform 20-minute budget per session, and scores determined exclusively by the harness-verified best metric.

Table 1 reports the performance of the pristine baseline, the shipped engine with self-improvement disabled (**RSI OFF**, `--improver noop`), the shipped engine with naive self-improvement enabled (**RSI ON**), and a strong reference baseline established by **Claude Code** (mean ± standard deviation over 3 independent 20-minute sessions).

The Assignment's Core Challenge. As shown in Table 1, the shipped naive RSI engine achieves virtually no improvement over the RSI OFF control (less than 2% difference; on several tasks, naive self-edits are reverted or even slightly detrimental). Meanwhile, Claude Code achieves substantially higher performance across all five tasks.

Your objective is twofold:

1. Demonstrate a statistically verifiable performance gap between your **RSI ON** system and its **RSI OFF** control.
2. Close as much of the performance gap to the strong Claude Code baseline as possible, through principled algorithmic strategies rather than hard-coded heuristics.

Strategy Development: What You Must Build

All strategy extension points are defined by Python `typing.Protocol` seams and marked with `TODO(homework)` throughout the codebase. You will implement concrete classes for these seams and wire them in `build_default_engine` (`rsi/core/engine.py`) or via configuration:

C — Core Strategies

- **C-1. Evidence-Guided Proposer (`rsi/core/proposer.py`):** The naive proposer walks a fixed grid and completely ignores past outcomes. You must build an intelligent proposer that:
  - Queries the knowledge platform (`KnowledgePlatform.evidence_for`) and ranks parameter modifications according to historical response sensitivities, uncertainty, and empirical gains.
  - Constructs adaptive value grids (e.g., local zooming around successful keeps, multi-scale sampling, or Bayesian search).
  - Avoids repeating discarded or failed parameters via memory mechanisms.
  - (Optional) Integrates with the LLM provider seam (`rsi.llm`) to compose candidate edits contextually.
- **C-2. Meaningful Self-Improver (`rsi/core/improver.py`):** The naive improver simply rotates a priority list when no-change waste exceeds 30%. Build a real self-improver that:
  - Analyzes detailed waste patterns in `EfficiencyReport` (noise floor traps, divergence collapses, wasted cycles per knob).
  - Explores the strategy search space within the sentinel region of `state/strategy.py` (tuning search widths, re-prioritizing exploration bands, or introducing new assignments).
  - Produces candidate self-edits that reliably survive guard checks and improve post-re-entry yield.
- **C-3. Guard Layer 4 Override Scan (`rsi/selfedit/guard.py`):** Implement an independent, deterministic scanner or judge that evaluates candidate mutable blocks and enforces default-reject security on suspicious code constructs.
- **C-4. Stage-2 Hardening and Meltdown Brake (`rsi/core/improver.py`):** Enhance self-edit arbitration with paired-rubric verification, degradation monitoring, and an automatic meltdown brake that halts self-modification after consecutive reverts or prolonged zero-yield stretches.

E — Evaluation Strategies

- **E-1. Benchmark Upgrader (`rsi/evaluation/benchmark.py`):** The naive upgrader merely increments noise seeds. Build an upgrader that progressively hardens benchmarks across epochs (e.g., increasing task difficulty, expanding condition regimes, evaluating against held-out splits) while properly managing cross-epoch incumbent re-baselining.
- **E-2. Verdict Gates (`rsi/evaluation/gates.py`):** Augment the gate chain with statistical significance tests against seed replication variance, shortcut/reward-hacking detection, and held-out validation gates.
- **E-3. Diagnostician (`rsi/evaluation/diagnostics.py`):** Replace sliding-window heuristics with robust estimators: noise floors derived from replication variance, fitted saturation curves, and calibrated recommendations (`continue`, `upgrade_benchmark`, or `saturate_question`).

K & R — Knowledge and Rigor

- **K-1. Question-Graph Evolution (`rsi/core/engine.py`):** Design principled slow-loop navigation over the typed edge vocabulary: decomposing saturated questions into refined sub-questions (`refines`), enforcing prerequisite chains (`depends_on`), constructing refutation trees (`challenges`), and transferring insights across questions.
- **R-1. Result Provenance (`rsi/tasks/runner.py`):** Implement cryptographic attestation sidecars verifying the SHA-256 digest of results, return codes, and editable files at execution time.
- **R-2. Custom Task Bundle (Optional Bonus):** Construct a new, self-contained machine learning task bundle adhering strictly to the `program.md` contract.

Grading Criteria and Assessment Guide

Your project submission will be evaluated on a 100-point scale across four core dimensions, as outlined in Table 2 (mirrored from the cover page of `submission/main.pdf`). Within each component, marks reflect the strength of verifiable empirical evidence: fully met (85–100%), met but thin (70–84%), asserted without adequate proof (60–69%), and unsupported (<60%).

Component	Weight	Evaluation Criteria
Literature Survey	10%	**Coverage:** Foundations of recursive self-improvement and automated research, plus recent related works. **Precision:** Every citation directly supports the claim it is attached to; no citation padding. **Synthesis:** Thematic comparison and contrasting of methods rather than a disconnected bibliography list. **Gap & Positioning:** Concrete identification of the technical bottleneck addressed, explicitly delineating what is inherited from the base harness versus what is newly proposed.
Reasoning & Insight	30%	**Diagnosis:** Bottlenecks (e.g., wasted cycles, unproductive self-edits) diagnosed from measured data, not intuition. **Motivation:** Clear hypothesis and causal rationale behind each strategy. **Course Linkage:** Rigorous application of machine learning principles: exploration–exploitation tradeoffs, bias–variance, credit assignment, multi-armed bandits/MDPs, out-of-distribution generalization, and statistical hypothesis testing. **Formalization:** Mathematical or algorithmic formulation of decision problems with stated assumptions. **Compute Awareness:** Sound compute/budget considerations and justification for rejected designs.
Experiments	30%	**Implementation:** All claimed strategies implemented in runnable code within the frozen engine and trust boundary. **Protocol:** Fair, matched baselines over equal compute budgets across multiple pre-registered seeds ($N \geq 3$, reported as mean $\pm$ standard deviation). **Scope:** Thorough evaluation across all five public tasks. **Offline Evaluation (Anti-Overfitting):** To prevent overfitting to the public benchmarks, the course staff will evaluate your submitted strategy implementations on **five unseen offline (held-out) tasks**; hard-coding task-specific shortcuts will be detected and penalized. **Reproducibility:** Exact commands, seed configurations, and ledgers provided; clean checkout cleanly reproduces numbers. **Honesty:** Full disclosure of negative results, regressions, and failed runs; no cherry-picking.
Result Analysis	30%	**Attribution:** Rigorous ablation studies isolating which specific strategy component drives performance gains. **Causal Mechanisms:** Explanations of gains and failures (e.g., noise overfitting, distribution shifts, exploration collapse). **Dynamics:** Quantitative analysis of keep-rate curves, saturation behavior, and per-cycle dynamics. **Self-Edit Anatomy:** Complete causal trace of at least one promoted self-edit (waste trigger $\rightarrow$ guard passage $\rightarrow$ observing-window validation $\rightarrow$ promotion $\rightarrow$ re-entry yield gain). **Calibration:** Candid discussion of limitations, compute overhead, and threats to validity.

Table 2: Project 2 assessment rubric (100 points total, accounting for 30% of final course grade).

Academic Integrity and Code Rules.

- **Plagiarism & Fabrication:** All reports and codebases will be processed through automated plagiarism detection tools. Fabricating experiment logs, altering baseline numbers, or manually doctoring results will result in an automatic score of zero and formal disciplinary action.
- **Frozen Scorer Integrity:** Modifying task scorers, hacking seed generation, or modifying verdict gates to inflate scores is strictly prohibited. Graded evaluations will run on pristine reference copies of the evaluation tasks.
- **LLM Usage Disclosure:** In accordance with the NeurIPS Paper Checklist (Item 16 in `appendix/a_checklist.tex`), you must transparently disclose any use of LLMs in strategy implementation or writing assistance.

Submission Guidelines and Package Structure

Your submission must be uploaded to Canvas as a single `.zip` archive before the deadline. The root of the unpacked archive must contain:

1. **Written Report (`submission/`):**
   - The compiled report `main.pdf` (at most 9 pages of body text, formatted according to the NeurIPS 2025 preprint template; references, NeurIPS checklist, and reproducibility appendix are free of the page count).
   - Complete, buildable LaTeX sources: `main.tex`, `references.bib`, vector figures in `figures/`, and all section files under `sections/` and `appendix/`.
2. **Source Code (`src/` or `rsi/`):** Complete, executable Python implementation of all your strategies wired into the engine. The existing test suite (`pytest tests/ -q`) must pass cleanly.
3. **Experimental Records (`experiments/` and `state/`):** Full execution records supporting your report: run logs (`runs.jsonl`), the self-edit ledger (`ledger.jsonl`), benchmark epoch logs (`epochs.json`), and generated session report directories. Graders will verify the provenance of every reported number.
4. **Replication Specification (`README.md` & `environment.yml`):** Clear step-by-step documentation detailing environment creation, random seed lists, and single-line reproduction commands for reproducing the headline results.

Figure and Plotting Conventions. All figures in your report must conform to professional academic standards:

- Figures must be saved in **vector PDF** format (`pdf.fonttype = 42`, `savefig.dpi = 120`). Rasterized screenshots are unacceptable.
- Typography must use **Times New Roman** / serif fonts matching the paper text, with legible font sizes (at least 7 pt at printed dimensions).
- Use an academic color palette (e.g., dark blue `#1f4e79`, purple `#7b1fae`, dark red `#b22222`, and silver/gray `#c0c0c0`).
- All trajectory and performance plots must depict empirical uncertainty (e.g., error bands or bars indicating mean $\pm$ standard deviation across ≥ 3 seeds).

Getting Started and Helpful Commands

Environment Setup

From the root of your project directory, set up the conda environment and run the test suite:

```
conda env create -f environment.yml
conda activate autoresearch-base
pytest tests/ -q
```

Quickstart Runs

Execute quick baseline trials on the demo and research bundles:

```
# Sub-second demo run on quadratic optimization
python -m rsi.core --task tasks/quad-optim --out out-demo --cycles 6 --seed 7

# Research run on the Iris out-of-distribution task
python -m rsi.core --task tasks/iris-ood --out out-iris --cycles 16 --seed 7

# Inspect learned knowledge and diagnostics
python -m rsi.knowledge --kb out-iris/kb frontier
python -m rsi.knowledge --kb out-iris/kb evidence
python -m rsi.evaluation --runs out-iris/state/runs.jsonl report
```

Generating Visual Reports and Ablation Comparisons

Generate full visual session reports and multi-seed comparisons using the provided analysis scripts:

```
# Generate visual report with RSI ON vs OFF trajectory comparisons
python scripts/report.py --session main=out-iris-on --session control=out-iris-off \
    --task iris-ood --metric ood_accuracy --direction max --dest report-iris

# Multi-seed statistical ablation (produces mean +/- SD curves)
python scripts/compare_seeds.py --task iris-ood --seeds 7,42,123 --cycles 24 \
    --dest experiments/ablation-iris
```

Building the Written Report

Navigate to the `submission/` directory and build your PDF report using `make`:

```
cd submission
make        # compiles main.pdf via pdflatex and bibtex
make clean  # cleans temporary build files
```

Before final submission, verify that all `[TODO:...]` annotations in your report have been resolved, confirm that your student ID and email appear in the author block, and perform a clean checkout test of your code archive.